Deep Learning Approach for Image-Based American Sign Language Recognition

Deepak Yadav
School of Computing
National College of Ireland
Dublin, Ireland
x21219991@student.ncirl.ie

Thi Thanh Thuy Nguyen
School of Computing
National College of Ireland
Dublin, Ireland
x22107720@student.ncirl.ie

Tulio Begena Araujo
School of Computing
National College of Ireland
Dublin, Ireland
x22133721@student.ncirl.ie

***Abstract*—This work presents a deep learning approach for image-based American Sign Language (ASL) recognition, focusing on enhancing communication for the deaf and hard-of-hearing communities. The primary contribution is a specialized Convolutional Neural Network (CNN) model built to find hidden patterns in data and utilize them to efficiently interpret sign language components. The model for gesture recognition was taking use of the ASL Alphabet dataset containing 8100 images and it is evaluated under three approaches, comparing two pre-processing methods for the images and no preprocessing. The findings also emphasize how pre-processing has a significant impact on performance. Notably, the proposed model achieves an impressive accuracy of 98% averagely in classifying ASL signs, highlighting the importance of deep learning and preprocessing for accurate ASL recognition. Overall, this study is an important step toward more accessible and equitable communication for various communities.**

***Index Terms*—Deep Learning, CNN, Sign Language, ASL, Image-based**

I. Introduction

Language familiarity and proficiency are crucial for effective communication [1]. When individuals express themselves, they use a language they are comfortable with. However, there are people who struggle with listening, speaking, or understanding languages [1]. Therefore, American Sign Language, a visual-gestural language, is critical to the linguistic and cultural identity of this community [2].

Advances in deep learning have transformed numerous sectors of computer vision in recent years, and one area that has seen tremendous progress is image-based American Sign Language (ASL) recognition [3]. This advanced technique has the potential to narrow communication barriers and increase inclusively for the deaf and hard-of-hearing groups, stated in various previous studies [3] [4].

The World Health Organization anticipates that by 2050, around 2.5 billion people will have some degree of hearing loss, with 700 million requiring rehabilitation services [5]. Failing to address this issue will lead to health consequences, exclusion from communication, education, and employment, and substantial financial losses. The prevalence of hearing loss is expected to increase by over 1.5 times in the next three decades, affecting more than 700 million individuals [6].

Hand gestures play a vital role in non-verbal communication, serving as a support to overcome linguistic boundaries or, in the case of people high degree of hearing loss, been the main method of communication [7] [8]. Recognizing and interpreting these gestures can open up new avenues for communication and accessibility. Deep learning, a subfield of artificial intelligence, has demonstrated remarkable success in various domains, including computer vision and natural language processing. Applying deep learning techniques, specifically a Convolutional Neural Network (CNN), to the recognition of hand gestures has the potential to revolutionize communication for individuals with hearing loss and promote inclusion [9]. As a result, the motivation of this project is to develop a model for ASL recognition system to facilitate communication for the deaf and hard of hearing people.

Research Question:

How can a CNN model be used to recognize and comprehend American Sign Language gestures in images?

By developing robust and accurate sign language recognition systems using CNN, we can overcome communication barriers and empower individuals with hearing loss. This research's objective is to contribute to the advancement of sign language recognition technology by leveraging the capabilities of deep learning algorithms. Through comprehensive experimentation and analysis, we seek to enhance the accuracy, efficiency, and usability of such systems, ultimately creating a more inclusive world where individuals with hearing impairments can fully participate in communication and society.

It is crucial to take action by promoting language accessibility, investing in rehabilitation services, and addressing the challenges faced by those with hearing loss. By doing so, we can create a more inclusive world, where communication barriers are minimized, and individuals have equal opportunities to thrive and contribute.

The rest of this report is structured as follows: Section II provides a thorough overview of the existing studies on ASL recognition. The proposed framework for the ASL learning application, including dataset description, methodology and the proposed framework will be described in Section III. Section IV displays the validation findings and analyzes the performance of the developed recognition system. Section V outlines the research's findings, including the conclusion, research contributions, limitation as well as future development.

II. Related Work

Sign language recognition is an important area of study that has the potential to improve the deaf and hard-of-hearing community's quality of life [10]. Numerous research investigations on sign language recognition have been conducted throughout the years, working with various approaches and with a variety of approaches to sign language detection, including the use of hand gestures, face expressions, and body movements [11]. These research projects focus not just on ASL [12] [13] [14], a widely used sign language, but also on other languages such as Filipino [15], Indian [16], Italian [17], and Turkish [18]. In recent years, deep learning has emerged as an approach for sign language identification, with various techniques being applied to both photos and videos [13] [9].

The most typical data collecting method is to utilize a standard video camera to capture hand photos from various people, angles, lighting, backdrops, and sizes [9]. While other researchers choose to use internet-sourced datasets.

Table I displays the recent studies on sign language recognition.

Regarding the type of neural network, convolutional neural network (CNN) is one of the most widely utilized deep learning techniques in sign language recognition due to its accuracy [9]. CNNs are particularly ideal for image recognition duties making them a good option for sign language identification [19]. A common CNN-based technique begins with preprocessing sign language videos to extract frames, which are then fed into the CNN to extract features. The features are then input into a classifier, such as a long short-term memory (LSTM) network, which recognizes the indications [9]. Significantly, a study of using CNN model for Arabic sign language recognition achieved impressively 99% accuracy on the test set [20]. However, the authors also suggested some drawbacks of using CNNs regarding contextual information limitation, because CNNs generally focus on existing features in input images and may fail to acquire the relevant contextual information, lower accuracy in complex signing contexts. This limitation can be seen in case of [21], where the model only reached 84% accuracy after a limited number of images were trained with 100 epochs. Furthermore, when characters have very high similarities between classes, the recognition performance of CNN systems may decrease [22].

The Long Short-Term Memory (LSTM) network is another deep learning technology that has been applied in sign language recognition [14]. LSTMs are a form of recurrent neural network (RNN) that excels at handling sequence data, especially sign language video. The sign language videos are transferred into the network, which learns to recognize the signs based on the sequence of frames in an LSTM-based manner [23]. In the recent study, [24] presented the LSTM model that achieves very high performance with 99.55% accuracy in Chinese Sign Language recognition.

Attention-based networks have also been attracted attention in the recognition of sign languages in recent researches [25]. Attention-based networks allow the network to focus on the most essential elements of the input, which is especially effective in sign language recognition since the vital information may be scattered across numerous frames. One of attention-based networks, the Sign Language Graph Time Transformer (SLGTformer) is currently the most successful non-ensemble method for modeling keypoint sequence motion dynamics [25]. It can be seen that this approach is able to handle a large amount of images obtained from video, however, the author's research did not discuss any specific computer power requirements.

Transformer networks have been presented as a solution to the restriction of CNN and LSTM in video frame sequence understanding [26]. The algorithms employ a self-attention method to better grasp the context, however past research indicates that Transformers' sequential learning ability is overly dependent on the ordering information it receives [27]. Notably, according to the findings of the study of [26], the complete Transformer model is greater than the CNN structure in sign language recognition tasks. Although CNN still wins Transformer in computer vision applications such as image classification tasks, it is more important for sign language recognition to handle high-level latent semantic information from images such as actions, facial expressions, and gestures.

Notably, capsule networks (CapsNet) are a viable alternative to convolutional neural networks (CNN), which consider an entity's spatial relationships and orientations. According to prior research, CapsNet has a stronger generalization and expressiveness capacity on unknown data than CNN dose, and its performance does not vary by different routing iterations [28]. Moreover, the experimental findings from [28] also show that CapsNet had a greater capacity for generalization and expressiveness on unseen input than CNN dosage. Meanwhile, CapsNet outperforms other classic deep learning approaches in the research of [29].

Deep Learning approaches have displayed significant prospects in recent years for sign language recognition. The most widely used strategy has been a combination of CNN and LSTM networks, but other deep learning architectures such as attention-based LSTM networks, transformer networks and capsule networks have also been investigated. On the American Sign Language (ASL) dataset, several techniques produced remarkable accuracies, with some approaches obtaining accuracies of 99% [14] [17] [36]. Deep learning in sign language recognition has the potential to improve the deaf and hard-of-hearing communities' quality of life by allowing them to communicate more effectively with the hearing community.

III. Methodology

The experimental research for sign language recognition shall employ an ASL dataset of 8100 images for model development. The input dataset will be under the pre-processing stage using three separate procedures before divided into training, evaluation and testing set. The training set shall be fed into CNN for model training and evaluated on the evaluation and testing sets. The accuracy of the chosen model will be taken into account for implementation.

TABLE I
RECENT STUDIES ON SIGN LANGUAGE RECOGNITION

Title (Year)	Language/Data type	Classifier	Chosen model and Accuracy
Deep Learning-Based Sign Language Digits Recognition From Thermal Images With Edge Computing System (2021) [30]	Thermal Imaging Dataset	CNN	CNN with 99.52%
Real-time Assamese Sign Language Recognition using MediaPipe and Deep Learning (2023) [31]	Assamese Sign Language	Custom Feedforward Neural Network	MediaPipe Feedforward Neural Network with 99%
American sign language recognition and training method with recurrent neural network (2021) [14]	ASL with 26 alphabets	Recurrent Neural Network (RNN)	RNN with 99.44%
Sign to Speech Convolutional Neural Network-Based Filipino Sign Language Hand Gesture Recognition System (2021) [32]	Filipino sign language of 20 hand gesture	CNN	CNN with 95%
Iraqi Sign Language Translator system using Deep Learning (2023) [20]	Arabic, video	CNN	CNN with 99%
Sign Language to Text Conversion using Deep Learning (2023) [21]	ASL	CNN	CNN, 84%
Finger Spelled Signs in Sign Language Recognition Using Deep Convolutional Neural Network (2021) [33]	ASL, image,	CNN	CNN, 98%
CNN based feature extraction and classification for sign language (2021) [22]	ASL of 36 characters	CNN, Support Vector Machine (SVM)	CNN, 99.82%
An Attention-Based Approach to Sign Language Recognition (2022) [25]	ASL, video, 2000 words	the Sign Language Graph Time Transformer	SLGTFORMER
Full transformer network with masking future for word-level sign language recognition (2022) [26]	ASL and CSL, video	CNN, Transformer, Self-Attention	Transformer
Sign language digits and alphabets recognition by capsule networks (2022) [28]	Sign language digits of (0–9) and alphabetic letters of (A–Z)	CapsNet	CapsNet 99.52 from Kaggle%
Egyptian Sign Language Recognition Using CNN and LSTM (2021) [34]	Egyptian, 9 characters	CNN, LSTM	CNN, 90%
A Modified LSTM Model for Chinese Sign Language Recognition Using Leap Motion (2022) [24]	Chinese, leap motion	LSTM	LSTM with 99.55%
American Sign Language Recognition for Alphabets Using MediaPipe and LSTM (2022) [35]	ASL, 26 alphabets by MediaPipe	LSTM	99%

Figure 1 depicts the process flow for carrying out the project, which will make up the CRoss Industry Standard Process for Data Mining (CRISP-DM) [37] for image-based sign language recognition tasks.

A. Dataset description

Various sign-language datasets are available open-source for hand-gesture recognition researchers. These databases differ in size, format, and illumination conditions. These datasets exhibit diversity not only in their scale, but also in data formatting nuances and the intricate interplay of lighting conditions. An outstanding contribution to this landscape arises from a dedicated collective. This group, characterized by their unwavering commitment, has meticulously assembled an extraordinary American Sign Language (ASL) dataset that spans across three distinct iterations: the raw, unprocessed embodiment; the judicious application of adaptive thresholding; and the nuanced integration of Canny edge detection.

The ASL dataset [38], emerged from the collaborative efforts of a skilled quintet. Each member of this adept team painstakingly captured a gallery of 60 photographs, encapsulating the graceful gestures representing the 26 alphabetic characters, alongside an eloquent void gesture. The culmination of this painstaking endeavor gave rise to an expansive repository boasting a staggering 8100 images. These images

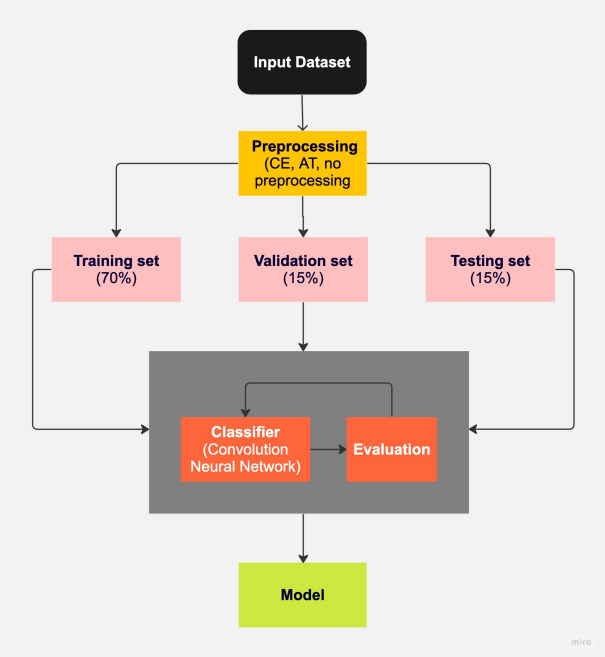

Fig. 1. Model development flowchart

were meticulously extracted from the gestural repertoire of five individuals. The division for training, validation, and testing purposes adheres to the ratios of 70%, 15%, and 15% respectively. The repository takes residence within a designated folder named "DATA".

A refined iteration of this dataset emerges through the application of the adaptive thresholding technique. Executed with finesse by the same adroit quintet, this evolution involves the judicious application of Gaussian Blur and adaptive thresholding on each of the 8100 images. The intent here is to amplify the discernibility of the intricate gestural contours. This specialized version of the dataset serves the purpose of model evaluation, shedding light on how the model interacts with preprocessed data unencountered during its training phase. This collection is housed within a folder christened "dataAT".

The onward progression of the dataset materializes with the infusion of the Canny edge detection preprocessing methodology. Once again, the steadfast team undertakes the endeavor of capturing 60 images for each gestural representation, aggregating to a grand total of 8100 images. The employment of the Canny edge detection technique lends an additional layer of finesse to the dataset. It is this very dataset that also assumes the role of evaluating the model's performance. It finds its abode within a folder labeled "dataCE".

B. Data Preprocessing

This research embarks on a thorough investigation into data preprocessing, a crucial phase in the research journey. As explained in Section III-A, the study revolves around three different datasets: a raw dataset, and two additional datasets that have undergone some adjustments using Canny edge (CE) detection and adaptive thresholding (AT) techniques.

An important part of this research is ensuring the quality of the data. To achieve this, a careful filtering process is applied, selecting only images with file extensions like ['jpeg', 'jpg', 'bmp', 'png']. This helps create a cleaner dataset for analysis by removing any unrelated files.

It's important to note that the dataset used in this research has already been prepared and processed before it's used. As a result, the research focuses on working with the prepared data for analysis and model development. This approach avoids the need for further adjustments. This emphasizes the significance of how the dataset has been prepared, allowing the research to proceed with a solid and well-organized starting point.

C. process model

In this study, the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology has been employed as a structured framework to address the research objective. The primary focus was on addressing the challenge of sign language image classification, aiming to enhance communication through more accurate models. Initially, an in-depth analysis of the dataset was conducted, laying the foundation for the subsequent steps.

The dataset was meticulously examined to gain insights into its characteristics and nuances. It was then leveraged to address the task of classifying sign language images effectively. Notably, the dataset arrived preprocessed, containing images that had undergone necessary transformations for analysis. Building upon this, a Convolutional Neural Network (CNN) model was constructed. CNNs are adept at learning hierarchical features from images, making them an ideal choice for image classification tasks.

The generated CNN model was systematically trained on the prepared dataset. To gauge its performance, rigorous evaluations were conducted using a distinct test dataset across all three designated data folders. This step ensured that the model's effectiveness was assessed comprehensively and provided insights into its generalization capabilities.

Through a process of hyperparameter tuning, involving techniques such as modifying the pool layers or adjusting filter counts within the CNN architecture, the model's performance was optimized. This meticulous tuning was instrumental in achieving a desirable level of accuracy, validating the effectiveness of the chosen approach.

After these iterative steps, during which the model underwent refinement and enhancement, a satisfactory level of accuracy was reached. This carefully tuned model, which now demonstrated a strong predictive capability, was deemed final for deployment. Thus, through the systematic application of the CRISP-DM framework, the study successfully tackled the research problem of sign language image classification, ultimately contributing to improved communication through more accurate and reliable models.

D. Frameworks and Tools

This section establishes the foundational framework and essential tools that underpin the investigation. It introduces the pivotal components that contribute to the technical infrastructure of the research, thereby facilitating a comprehensive and rigorous exploration.

The pivotal frameworks and tools discussed in this section encompass:

- **Google Colab:** An advanced cloud-based Jupyter Notebook platform known for its seamless code execution and robust GPU/TPU acceleration. This platform serves as an environment for coding, experimenting, and data analysis.
- **TensorFlow:** An open-source machine learning framework that plays a central role in the development and training of intricate models. Its flexibility and comprehensive ecosystem empower researchers to design and evaluate complex algorithms.
- **Keras:** An integral high-level neural networks API, seamlessly integrated into TensorFlow's architecture. Keras simplifies the construction of neural networks and enables efficient experimentation with different model architectures.

This preliminary exposition delves into the core components, each with its specific role in the research's technical journey. Moreover, the following key libraries further enhance the research's capabilities:

- **File and Data Management:** The `os`, `zipfile`, and `shutil` modules streamline file and directory oper-

ations, contributing to effective data management and organization.

- **Numerical Computing:** Leveraging the `numpy` library, mathematical computations and flexible data manipulation are facilitated, forming a fundamental basis for various analysis processes.
- **Image Processing:** Utilizing the `cv2` module from OpenCV, the research benefits from versatile capabilities for image processing tasks, enhancing data preprocessing and feature extraction.
- **Deep Learning Framework:** The `tensorflow` framework enables the creation, training, and deployment of intricate machine learning models, substantiating the research's technical depth.
- **Interactive Visualization:** Employing the `plotly` and `plotly.graph_objects` libraries, the research incorporates interactive visualizations, fostering a dynamic and comprehensive representation of data.
- **Model Evaluation:** The `sklearn.metrics` library provides a comprehensive suite of functions for assessing model performance, including crucial metrics such as precision, recall, accuracy, and confusion matrix.
- **Neural Network Construction:** The `keras.models` and `keras.layers` modules facilitate the construction and specification of neural network architectures, streamlining the implementation of complex models.
- **Data Visualization:** Leveraging the `matplotlib.pyplot` module, the research generates informative and insightful data visualizations, enhancing the analysis and interpretation of results.

In Summary, this comprehensive ensemble of frameworks and tools establishes a robust and sophisticated technical foundation. This foundation, in turn, empowers the research to seamlessly navigate through data manipulation, model development, evaluation, and visualization, fostering an informed and research.

E. Model Architecture

The adopted model architecture serves as a prominent illustration of a deep learning framework tailored specifically for image classification assignments. The arrangement of the model takes on a sequential structure, wherein various layers are systematically stacked, with each layer assuming a crucial function in extracting intricate features from the input images. The visual representation of the model summary can be observed in Figure 2. This summary encapsulates the configuration and parameters of the model, offering a concise overview of its architecture. The model architecture for this study is illustrated on Figure 3, which also referred to the construction from [20] but with a lesser input fed.

The architecture commences with a `Conv2D` layer equipped with 32 filters of dimensions 3×3. This layer, known for its feature extraction capability, analyzes the image through a convolution operation and generates an output volume with dimensions $(\text{None}, 254, 254, 32)$. Subsequently, a `MaxPooling2D` layer, which employs a 2×2 pooling

```
Model: "sequential"
_________________________________________________________________
 Layer (type)                Output Shape              Param #
=================================================================
 conv2d (Conv2D)             (None, 254, 254, 32)      896

 max_pooling2d (MaxPooling2D  (None, 127, 127, 32)     0
 )

 conv2d_1 (Conv2D)           (None, 125, 125, 16)      4624

 max_pooling2d_1 (MaxPooling  (None, 62, 62, 16)       0
 2D)

 conv2d_2 (Conv2D)           (None, 60, 60, 32)        4640

 max_pooling2d_2 (MaxPooling  (None, 30, 30, 32)       0
 2D)

 flatten (Flatten)           (None, 28800)             0

 dense (Dense)               (None, 256)               7373056

 dense_1 (Dense)             (None, 27)                6939

=================================================================
Total params: 7,390,155
Trainable params: 7,390,155
Non-trainable params: 0
_________________________________________________________________
```

Fig. 2. CNN Model Summary

mechanism, reduces the spatial dimensions by half, yielding an output of dimensions $(\text{None}, 127, 127, 32)$.

The process is reiterated with `Conv2D` Layer 2, housing 16 filters of size 3×3, and its subsequent `MaxPooling2D` Layer 2, achieving further dimension reduction to $(\text{None}, 62, 62, 16)$. `Conv2D` Layer 3, featuring 32 filters of dimensions 3×3, followed by `MaxPooling2D` Layer 3, generates an output volume with dimensions $(\text{None}, 30, 30, 32)$.

A `Flatten` layer transitions the output into a 1D vector of length 28800, preparing it for `Dense` Layer 1. This dense layer, housing 256 neurons and employing the rectified linear unit (ReLU) activation function, introduces non-linearity, allowing the model to discern complex relationships within the data.

At the end, the model reaches its conclusion with the `Dense` Layer 2, which serves as the output layer. This layer comprises 27 neurons, symbolizing the different classes used for classification purposes. Through the use of the softmax activation function, the outputs are converted into probabilities for each class, allowing the model to predict the most likely class for a given input.

The total count of trainable parameters, amounting to 7,390,155, emphasizes the complexity and capability of the architecture. This number reflects how intricate the model's inner workings are. The architecture is composed in a sequence, starting with convolutional and pooling layers, followed by dense layers. This arrangement mimics the traditional setup of deep convolutional neural networks, which are well-suited for detecting complex patterns in image data.

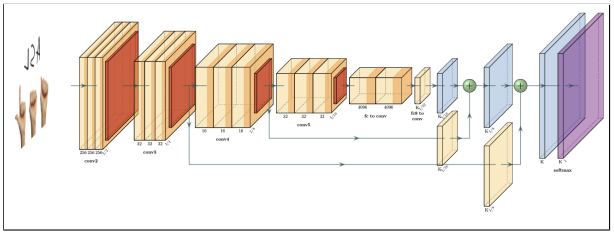

Fig. 3. The proposed CNN's architecture

F. Evaluation

The input dataset was splitted into separate training, validating, and testing subsets with the cutoff rates of 70%, 15%, and 15% respectively. This partitioning facilitated a comprehensive evaluation of the developed model. Particularly, the model assessment will involve the ultilization of confusion matrix with four key indicators including True Positive (TP), True Negative (TN), False Positive (FP), and False Negative (FN). Additionally, the accuracy, precision, recall, and F_1 metrics were employed to asses the model's effectiveness.

IV. Results

Different models were trained with the referred architecture. The difference between then was the data used as input. Using dataRAW as input, the modelRAW was trained. The same is valid for modelAT and modelCE. One more model was trained, modelFULL, that had the three types of images as input. Moreover, each model was evaluated against test data separated by the type of image preprocessing. Only the modelFULL was tested against a test data that comprehends all types of images together. These steps were important to scrutinize the hypothesis stating that the model trained with only one of the types of available images would perform good enough in any type of test data.

After loading the data, some images were displayed to verify its content. A sample of these images without any processing can be seen in Figure 4. The images differ in illuminating condition one from another. On the other hand, the hands are clearly distinguished from the neutral background, exacerbating that the dataset used is of high quality.

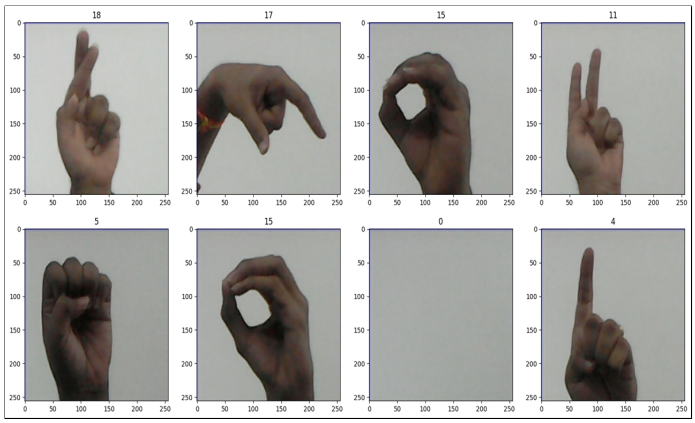

Fig. 4. Sample of images from dataset with respective labels

As can be seen in Figure 5, the models were trained with 5 epochs. Except for modelFULL, the models reached 99% accuracy at least from third epoch. The evolution of loss and accuracy of all models follow same pattern. An example of this evolution is on Figure 6, which is the evolution of loss and accuracy versus the number of epochs. The graph shows that there has been a sharp rise in the accuracy from the first to second epoch.

```
histRAW = modelRAW.fit(trainRAW, epochs=5, validation_data=valRAW, callbacks=[tensorboard_callback])

Epoch 1/5
177/177 [==============================] - 47s 189ms/step - loss: 0.9478 - accuracy: 0.7378 - val_loss: 0.1227 - val_accuracy: 0.9688
Epoch 2/5
177/177 [==============================] - 25s 142ms/step - loss: 0.0710 - accuracy: 0.9804 - val_loss: 0.0517 - val_accuracy: 0.9877
Epoch 3/5
177/177 [==============================] - 23s 130ms/step - loss: 0.0195 - accuracy: 0.9954 - val_loss: 0.0273 - val_accuracy: 0.9951
Epoch 4/5
177/177 [==============================] - 23s 131ms/step - loss: 0.0118 - accuracy: 0.9970 - val_loss: 0.0245 - val_accuracy: 0.9942
Epoch 5/5
177/177 [==============================] - 22s 123ms/step - loss: 0.0092 - accuracy: 0.9984 - val_loss: 0.0533 - val_accuracy: 0.9910

histAT = modelAT.fit(trainAT, epochs=5, validation_data=valAT, callbacks=[tensorboard_callback])

Epoch 1/5
177/177 [==============================] - 23s 121ms/step - loss: 0.6593 - accuracy: 0.8317 - val_loss: 0.1656 - val_accuracy: 0.9638
Epoch 2/5
177/177 [==============================] - 19s 108ms/step - loss: 0.0394 - accuracy: 0.9915 - val_loss: 0.0962 - val_accuracy: 0.9844
Epoch 3/5
177/177 [==============================] - 20s 112ms/step - loss: 0.0134 - accuracy: 0.9965 - val_loss: 0.0966 - val_accuracy: 0.9836
Epoch 4/5
177/177 [==============================] - 22s 124ms/step - loss: 0.0102 - accuracy: 0.9972 - val_loss: 0.1499 - val_accuracy: 0.9712
Epoch 5/5
177/177 [==============================] - 19s 106ms/step - loss: 0.0105 - accuracy: 0.9972 - val_loss: 0.0814 - val_accuracy: 0.9934

histCE = modelCE.fit(trainCE, epochs=5, validation_data=valCE, callbacks=[tensorboard_callback])

Epoch 1/5
177/177 [==============================] - 24s 116ms/step - loss: 0.4312 - accuracy: 0.8902 - val_loss: 0.0363 - val_accuracy: 0.9918
Epoch 2/5
177/177 [==============================] - 19s 107ms/step - loss: 0.0233 - accuracy: 0.9929 - val_loss: 0.0492 - val_accuracy: 0.9885
Epoch 3/5
177/177 [==============================] - 18s 102ms/step - loss: 0.0065 - accuracy: 0.9989 - val_loss: 0.0019 - val_accuracy: 0.9992
Epoch 4/5
177/177 [==============================] - 22s 124ms/step - loss: 7.4725e-05 - accuracy: 1.0000 - val_loss: 0.0018 - val_accuracy: 0.9984
Epoch 5/5
177/177 [==============================] - 21s 116ms/step - loss: 2.5998e-05 - accuracy: 1.0000 - val_loss: 0.0017 - val_accuracy: 0.9992

histFULL = modelFULL.fit(trainFULL, epochs=5, validation_data=valFULL, callbacks=[tensorboard_callback])

Epoch 1/5
533/533 [==============================] - 61s 111ms/step - loss: 0.6113 - accuracy: 0.8505 - val_loss: 0.3191 - val_accuracy: 0.9041
Epoch 2/5
533/533 [==============================] - 55s 103ms/step - loss: 0.0867 - accuracy: 0.9825 - val_loss: 0.0598 - val_accuracy: 0.9775
Epoch 3/5
533/533 [==============================] - 72s 135ms/step - loss: 0.0777 - accuracy: 0.9845 - val_loss: 0.0993 - val_accuracy: 0.9707
Epoch 4/5
533/533 [==============================] - 55s 102ms/step - loss: 0.0503 - accuracy: 0.9885 - val_loss: 0.0651 - val_accuracy: 0.9822
Epoch 5/5
533/533 [==============================] - 55s 102ms/step - loss: 0.0558 - accuracy: 0.9869 - val_loss: 0.0572 - val_accuracy: 0.9838
```

Fig. 5. Training history of the models

Generally, the accuracy of validation set increased from one epoch to another. There were not improvements after epoch five for the accuracy achieved for validation set, suggesting that the small improvements spotted regarding the accuracy on train set would cause overfitting of the models. The models modelRAW, modelAT, and modelCE reached more than 99% accuracy. The model modelFULL had 98.4% accuracy. These results may be linked to the fact that the last model was trained and validate with a more varied types of data, making it more complex to predict the classes. Nevertheless, all these initial results imply that the architecture used to build the models is adequate.

A more complete evaluation of the models were performed when testing each model against different types of test data available. The Table II summarize the accuracy values returned from the evaluations. In the same way, the Tables III and IV show the precision and recall, respectively. A similarity shared between these three tables is that not only the highest by a small margin, but the values above 98% are in the diagonals. The results from modelFULL are different from others, so they will be discussed separately. Outside the diagonals the values are as bad as 0.09% of precision when applying the modelCE on dataRAW data and reach a maximum of 80.74% of precision crossing modelRAM and dataAT. Another point is that the results for recall and accuracy are identical. Hence, and adding the fact that all classes are equally important, recall will not be discussed any further.

A confusion matrix was plotted for each pair of model and test data. Those low values discussed in the previous paragraph are for the pairs modelRAW versus dataAT and dataCE, modelAT versus dataRAW and dataCE, and modelCE versus dataRAW and dataAT. Figure 7 show the matrices for these pairs. The confusion matrices of the pairs modelRAW

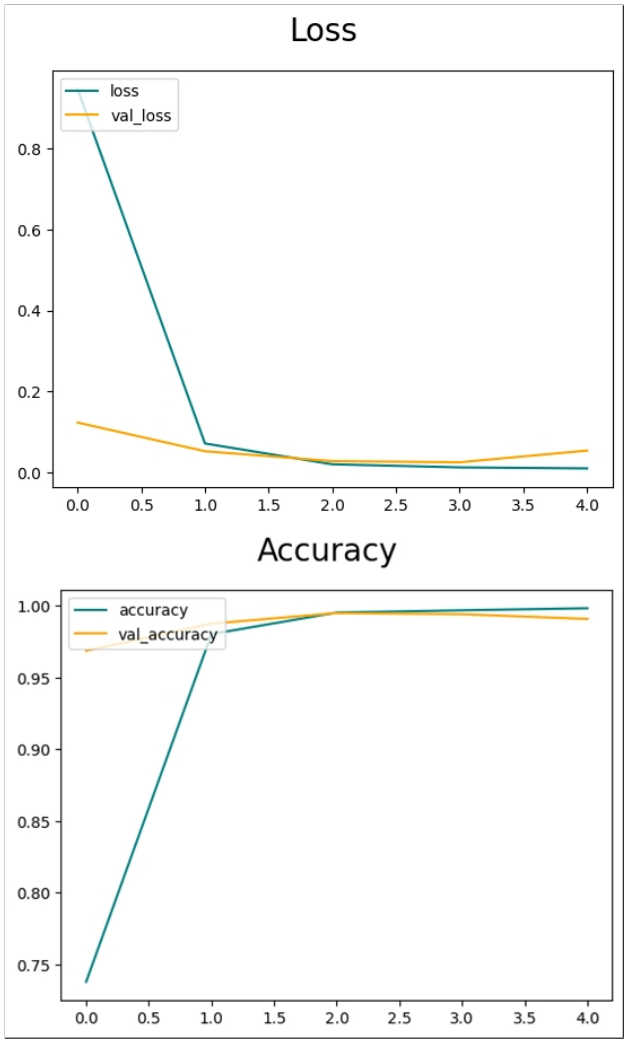

Fig. 6. Loss and accuracy over epochs for the model modelRAW

TABLE II
PRECISION OF THE MODELS FOR EACH TEST DATA

Precision (%)	modelRAW	modelAT	modelCE	modelFULL
dataRAW	98.62	61.39	00.09	87.61
dataAT	80.74	99.60	01.28	99.47
dataCE	54.09	45.53	99.76	98.95
dataFULL	-	-	-	98.76

versus dataRAW, modelAT versus dataAT, and modelCE versus dataCE are in Figure 8. The findings of this study suggest that the models that are trained with only one type of image, with or without a pre-treatment, perform poorly when tested with data containing images with a different treatment.

The last two matrices in Figure 7 are from the model modelCE. This model was not capable to distinguish the classes from the images without pre-treatment or with adaptive thresholding, classifying most of the images as none. This generated a vertical line in the matrix while the best result would be a distinct diagonal. This model returned the lowest values regarding precision, recall, and accuracy, as it is showed

TABLE III
RECALL OF THE MODELS FOR EACH TEST DATA

Recall (%)	modelRAW	modelAT	modelCE	modelFULL
dataRAW	98.52	51.15	02.96	81.99
dataAT	62.25	99.59	04.69	99.42
dataCE	49.42	42.85	99.75	98.85
dataFULL	-	-	-	98.71

TABLE IV
ACCURACY OF THE MODELS FOR EACH TEST DATA

Accuracy (%)	modelRAW	modelAT	modelCE	modelFULL
dataRAW	98.52	51.15	02.96	81.99
dataAT	62.25	99.59	04.69	99.42
dataCE	49.42	42.85	99.75	98.85
dataFULL	-	-	-	98.71

in its respective tables. This data reported here appear to support the assumption that since modelCE (Canny edge detection preprocessing method) was trained with more simple images, i.e., images containing just some edges as lines, it was not capable to assess more complex images.

Turning now to the results with good evaluation, they show clearly the diagonals in Figures 8. Between these three models, the modelRAW exhibited approximately 1% less precision and accuracy compared to modelAT and modelCE. The modelCE performed slightly better than modelAT considering the measured metrics.

With respect to modelFULL, it exhibited much more generalization power than previous models. Obviously due to the fact this model was trained with these three types of images. Its accuracy when tested against dataRAW was 82%. 99.4% against dataAT and 98.9% against dataCE. The confusion matrices regarding these pairs are shown in Figure 9. A possible explanation for these numbers might be that for a generic model it is easy to classify images with some kind of preprocessing technique than raw data.

The modelFULL predicted the classes from dataAT with less than 0.2% difference in accuracy than modelAT. Additionally, the modelFULL, compared to modelCE while testing against dataCE, produced an accuracy just 0.1% less. For unpreprocessed data, modelFULL, with its 82% accuracy, showed much more useful than modelAT and modelCE, which showed 2.96% and 51.15%, respectively. The performance trade-offs seems to be insignificant. A valid trade-off, however, could be the amount of computing power necessary to train the model. This model needed three times more data and time to be trained than the others.

For the non-preprocessed data, it is possible to note in the first matrix of Figure 9 that many errors occur when the model classify images as ”A” or ”N”, while the real label is for another class. Both letters are made with a more closed hand, in the American Sign Language. This could have caused the parameters of the model not distinguish these classes clearly from others. Although this problem do not appear when using test data images with the pretreatments explored in this study.

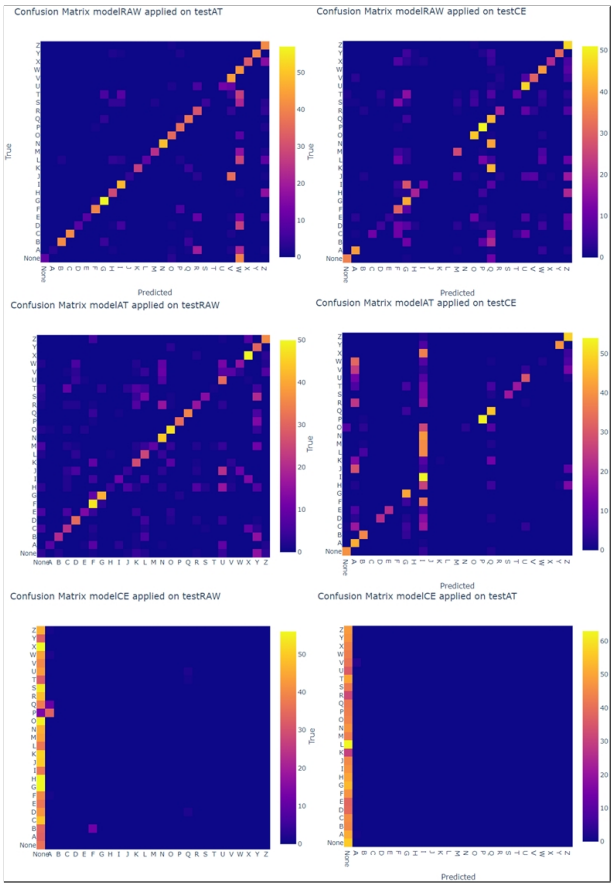

Fig. 7. Confusion matrices of the low evaluation values pairs model test data

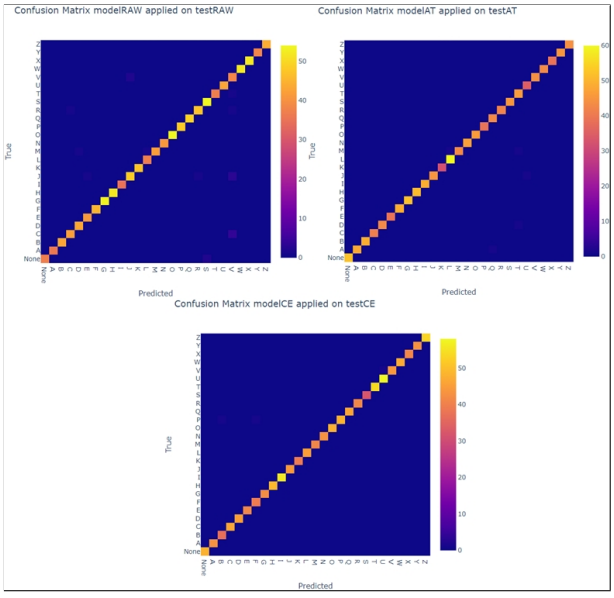

Fig. 8. Confusion matrices of the high evaluation values pairs model test data

V. Conclusion and Future Work

In this study, different image preprocessing techniques were avail regarding how it would affect Convolutional Neural Network models built to predict the class (letters) in American Sign Language images dataset. Different models were trained using raw data as input, and also with Canny edge detection, adaptive thresholding technique, and one more model with all these types of images as input.

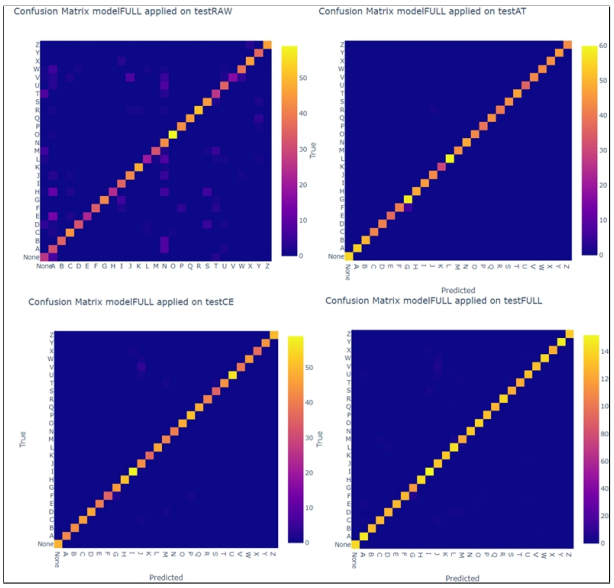

Fig. 9. Confusion matrices of the modelFULL

Firstly, the preprocessing technique used defined how well a model would predict the classes of test data. The only model capable to predict any type of data was the more generic model, named modelFULL. This model exhibited insignificant trade-off regarding performance, evaluated by accuracy and precision, compared to specific models. On the other side, this model is three times more computing power consuming than the others. If this is not a limitation of a project, the more generalist model should be applied.

Some classes, namely “A” and “N”, presented more difficulty for the generalist model to assign correctly. The evidence from this study suggests that the use of adaptive thresholding technique on a possible raw image dataset is capable to solve this problem and to increase the accuracy of predictions, making it recommended before the model is applied for classification.

Finally, according to the presenting findings, a more diverse data builds a best model. Hence, the use of more varied data is encouraged to train models with similar goal, i.e., to classify images.

For future works, more types of image preprocessing techniques could be analysed. Moreover, these techniques should be used to build a more diverse input data. Another improvement in this field could be the use of more polluted datasets. In other words, datasets with different backgrounds and illuminating conditions. This could lead to a even more generic model, possibly capable to classify unseen data.

REFERENCES

[1] Muhamad Alif Haji Sismat et al. The language of pandemic and its significance in effective communication. *Journal of Humanities and Social Sciences Studies*, 3(4):54–60, 2021.

[2] Tamara Keck and Keith Wolgemuth. American sign language phonological awareness and english reading abilities. *Sign Language Studies*, 20(2):334–354, 2020.

[3] Muhammad Al-Qurishi, Thariq Khalid, and Riad Souissi. Deep learning for sign language recognition: Current techniques, benchmarks, and open issues. *IEEE Access*, 9:126917–126951, 2021.

[4] Gero Strobel, Thorsten Schoormann, Leonardo Banh, and Frederik Möller. Artificial intelligence for sign language translation–a design science research study. *Communications of the Association for Information Systems*, 52(1):33, 2023.

[5] World Health Organization et al. Addressing the rising prevalence of hearing loss. 2018.

[6] World Health Organization. *World report on hearing: executive summary*. World Health Organization, Geneva, 2021. Licence: CC BY-NC-SA 3.0 IGO.

[7] Michael Argyle. Non-verbal communication in human social interaction. *Non-verbal communication*, 2, 1972.

[8] Amber Saleem, Samina Rana, and Marriam Bashir. Role of non-verbal communication as a supplementary tool to verbal communication in esl classrooms: A case study. *Propel Journal of Academic Research*, 2(2):39–52, 2022.

[9] Andra Ardiansyah, Brandon Hitoyoshi, Mario Halim, Novita Hanafiah, and Aswin Wibisurya. Systematic literature review: American sign language translator. *Procedia Computer Science*, 179:541–549, 2021. 5th International Conference on Computer Science and Computational Intelligence 2020.

[10] Patricia Elizabeth Spencer and Marc Marschark. *Advances in the spoken-language development of deaf and hard-of-hearing children*. Oxford University Press, 2005.

[11] Razieh Rastgoo, Kourosh Kiani, and Sergio Escalera. Sign language recognition: A deep survey. *Expert Systems with Applications*, 164:113794, 2021.

[12] KP Nimisha and Agnes Jacob. A brief review of the recent trends in sign language recognition. In *2020 International Conference on Communication and Signal Processing (ICCSP)*, pages 186–190, 2020.

[13] Anirudh Tunga, Sai Vidyaranya Nuthalapati, and Juan Wachs. Pose-based sign language recognition using gcn and bert. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 31–40, 2021.

[14] Carman KM Lee, Kam KH Ng, Chun-Hsien Chen, Henry CW Lau, SY Chung, and Tiffany Tsoi. American sign language recognition and training method with recurrent neural network. *Expert Systems with Applications*, 167:114403, 2021.

[15] Mark Benedict D. Jarabese, Charlie S. Marzan, Jenelyn Q. Boado, Rushaine Rica Mae F. Lopez, Lady Grace B. Ofiana, and Kenneth John P. Pilarca. Sign to speech convolutional neural network-based filipino sign language hand gesture recognition system. In *2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC)*, pages 147–153, 2021.

[16] Kaushal Goyal et al. Indian sign language recognition using mediapipe holistic. *arXiv preprint arXiv:2304.10256*, 2023.

[17] Arpita Halder and Akshit Tayade. Real-time vernacular sign language recognition using mediapipe and machine learning. *Journal homepage: www. ijrpr. com ISSN*, 2582:7421, 2021.

[18] Zekeriya Katılmış and Cihan Karakuzu. Elm based two-handed dynamic turkish sign language (tsl) word recognition. *Expert Systems with Applications*, 182:115213, 2021.

[19] S Mohamed Hussain Kani, S Abdullah, U Sheik Amanullah, and G Divya. Sign language recognition with convolutional neural network. *International Journal of Research in Engineering, Science and Management*, 6(5):116–119, 2023.

[20] rajaa mohammed and Suhad M. Kadhem. Iraqi sign language translator system using deep learning. *Al-Salam Journal for Engineering and Technology*, 2(1):109–116, Jan. 2023.

[21] Aruna Bhat, Vinay Yadav, Vishesh Dargan, and Yash. Sign language to text conversion using deep learning. In *2022 3rd International Conference for Emerging Technology (INCET)*, pages 1–7, 2022.

[22] Abul Abbas Barbhuiya, Ram Kumar Karsh, and Rahul Jain. Cnn based feature extraction and classification for sign language. *Multimedia Tools and Applications*, 80(2):3051–3069, 2021.

[23] Jai Joshi, Parshav Gandhi, and Rupali Sawant. Sign language certification platform with action recognition using lstm neural networks. In *2022 International Conference on Computing, Communication, Security and Intelligent Systems (IC3SIS)*, pages 1–6. IEEE, 2022.

[24] Bixiao Wu, Zhenyu Lu, and Chenguang Yang. A modified lstm model for chinese sign language recognition using leap motion. In *2022 IEEE International Conference on Systems, Man, and Cybernetics (SMC)*, pages 1612–1617. IEEE, 2022.

[25] Neil Song. Slgtformer: An attention-based approach to sign language recognition. *arXiv preprint arXiv:2212.10746*, 2022.

[26] Yao Du, Pan Xie, Mingye Wang, Xiaohui Hu, Zheng Zhao, and Jiaqi Liu. Full transformer network with masking future for word-level sign language recognition. *Neurocomputing*, 500:115–123, 2022.

[27] Neena Aloysius, Geetha M, and Prema Nedungadi. Incorporating relative position information in transformer-based sign language recognition and translation. *IEEE Access*, 9:145929–145942, 2021.

[28] Hongwang Xiao, Yun Yang, Ke Yu, Jiao Tian, Xinyi Cai, Usman Muhammad, and Jinjun Chen. Sign language digits and alphabets recognition by capsule networks. *Journal of Ambient Intelligence and Humanized Computing*, pages 1–11, 2022.

[29] Jia Lu, Minh Nguyen, and Wei Qi Yan. Sign language recognition from digital videos using deep learning methods. In *Geometry and Vision: First International Symposium, ISGV 2021, Auckland, New Zealand, January 28-29, 2021, Revised Selected Papers 1*, pages 108–118. Springer, 2021.

[30] Daniel S. Breland, Simen B. Skriubakken, Aveen Dayal, Ajit Jha, Phaneendra K. Yalavarthy, and Linga Reddy Cenkeramaddi. Deep learning-based sign language digits recognition from thermal images with edge computing system. *IEEE Sensors Journal*, 21(9):10445–10453, 2021.

[31] Jyotishman Bora, Saine Dehingia, Abhijit Boruah, Anuraag Anuj Chetia, and Dikhit Gogoi. Real-time assamese sign language recognition using mediapipe and deep learning. *Procedia Computer Science*, 218:1384–1393, 2023. International Conference on Machine Learning and Data Engineering.

[32] Mark Benedict D Jarabese, Charlie S Marzan, Jenelyn Q Boado, Rushaine Rica Mae F Lopez, Lady Grace B Ofiana, and Kenneth John P Pilarca. Sign to speech convolutional neural network-based filipino sign language hand gesture recognition system. In *2021 International Symposium on Computer Science and Intelligent Controls (ISCSIC)*, pages 147–153. IEEE, 2021.

[33] Reshmi Rajendran and Sangeeta Tulasi Ramachandran. Finger spelled signs in sign language recognition using deep convolutional neural network. *International Journal of Research in Engineering, Science and Management*, 4(6):249–253, 2021.

[34] Ahmed Elhagry and Rawan Glalal Elrayes. Egyptian sign language recognition using cnn and lstm. *arXiv preprint arXiv:2107.13647*, 2021.

[35] B Sundar and T Bagyammal. American sign language recognition for alphabets using mediapipe and lstm. *Procedia Computer Science*, 215:642–651, 2022.

[36] Daniel S Breland, Simen B Skriubakken, Aveen Dayal, Ajit Jha, Phaneendra K Yalavarthy, and Linga Reddy Cenkeramaddi. Deep learning-based sign language digits recognition from thermal images with edge computing system. *IEEE Sensors Journal*, 21(9):10445–10453, 2021.

[37] Christoph Schröer, Felix Kruse, and Jorge Marx Gómez. A systematic literature review on applying crisp-dm process model. *Procedia Computer Science*, 181:526–534, 2021. CENTERIS 2020 - International Conference on ENTERprise Information Systems / ProjMAN 2020 - International Conference on Project MANagement / HCist 2020 - International Conference on Health and Social Care Information Systems and Technologies 2020, CENTERIS/ProjMAN/HCist 2020.

[38] Karthik Sajjan Vanga Manoj Sahit Reddy Kuppa Venkata Krishna Paanchajanya, Karusala Deepak Chowdary and Killadi Venkata Jai Santeswar. American Sign Language (ASL). Kaggle.com, 2023. https://www.kaggle.com/datasets/krishnapaanchajanya/american-sign-language-asl.